

ClickSafe Project Details

Project Snapshot

ClickSafe is an AI-powered phishing-risk console. A user submits a URL before visiting it; ClickSafe validates the destination, visits it in an automated browser, gathers technical and reputation evidence, and returns a **Safe**, **Suspicious**, or **Malicious** verdict with a risk score from 0 to 100.

Implementation status: **Phases 1 through 7 are complete.** Phase 8, production hardening, is the remaining committed phase in the current roadmap.

ClickSafe is an evidence-assisted security tool. Its verdict is a risk assessment, not a guarantee that a website is harmless.

Implementation note: analysis payloads currently store `lifecycle.phase: 6`, which identifies the completed backend pipeline. The project-level Phase 7 dashboard integration is complete; the internal lifecycle marker can be aligned during Phase 8 housekeeping.

Product Goals

- Help users make a safer decision before opening an unfamiliar link.
- Combine browser, technical, reputation, and AI evidence rather than relying on one signal.
- Preserve enough evidence for a user or security analyst to understand the verdict.
- Keep the system modular so analyzers and data providers can be changed independently.
- Provide a responsive dashboard that works in light and dark modes.

Current User Flow

1. A user enters an HTTP/HTTPS URL or a bare domain in the React dashboard.
2. The frontend sends the URL to `POST /api/v1/analyze`.
3. The backend validates and normalizes the URL, then creates an analysis job in SQLite.
4. Playwright loads the destination in a new Chromium browser context.
5. ClickSafe records the redirect chain, final URL, response status, page HTML, and a full-page screenshot.
6. Technical analyzers and reputation providers collect evidence.
7. The evidence bundle is assessed with the OpenAI Responses API when an API key is configured. A clearly labeled local heuristic fallback is used when it is not.
8. The job, verdict, score, explanation, and evidence are persisted and shown in the dashboard.

Implemented Features

URL Validation and Job Lifecycle

- Accepts HTTP and HTTPS URLs, including bare domains such as `example.com`.
- Rejects unsupported schemes such as `ftp`.
- Enforces a request URL length between 1 and 2,048 characters.
- Normalizes host casing, removes fragments, removes default ports, and adds HTTPS to a bare domain.
- Persists the job lifecycle as `requested`, `running`, `completed`, or `failed`.

- Returns useful persisted failure information for validation and browser-navigation errors.

Browser Evidence Collection

- Uses Playwright and Chromium in headless mode by default.
- Opens each URL in a fresh browser context.
- Follows navigation and records HTTP redirect URLs, locations, and status codes.
- Records the final resolved URL and final response status.
- Saves a full-page PNG screenshot per analysis.
- Saves downloaded HTML with configurable byte limits and truncation metadata.
- Applies configurable browser navigation timeouts and redirect limits.

Technical Analyzers

Each analyzer is isolated in its own backend module and contributes standardized evidence items.

Analyzer	Evidence collected
Redirect analyzer	Redirect count, URL transitions, cross-host redirects, and related risk signals.
HTML analyzer	Page structure and potentially unusual or limited HTML content.
Metadata analyzer	Titles, descriptions, canonical data, and relevant page metadata.
Forms analyzer	Forms, credential-oriented fields, methods, actions, and external form targets.
JavaScript analyzer	Inline scripts and suspicious JavaScript indicators.
DNS analyzer	DNS resolution information for the destination hostname.
SSL analyzer	HTTPS certificate connectivity and certificate details.
WHOIS analyzer	Registered-domain information and age-related evidence where available.
Reputation analyzer	VirusTotal and Google Safe Browsing outcomes, isolated from technical evidence.

Reputation Services

- VirusTotal URL reputation integration using the v3 URL report endpoint.
- Google Safe Browsing v4 URL lookup integration.
- Independent provider execution so one failed provider does not discard the other provider's result.
- Configurable provider timeout handling.
- Explicit evidence for disabled API keys, malformed responses, HTTP failures, and timeouts.
- Reputation checks use the browser's final URL when one is available.

Google Safe Browsing v4 is intended for non-commercial use. A commercial deployment should use Google Web Risk or another appropriately licensed provider.

AI Verdict Generation

- Integrates the OpenAI Responses API through an async client.
- Sends a normalized, collected evidence bundle to the configured model.
- Uses strict JSON-schema structured output for predictable application data.

- Produces:
 - Verdict: **Safe**, **Suspicious**, or **Malicious**
 - Risk score: integer from 0 to 100
 - Explanation
 - Recommended user action
 - Confidence score
 - Weighted evidence rationale
- Transparently reports provider name, model, response ID, completion status, and whether a fallback was used.
- Uses a conservative local heuristic when **OPENAI_API_KEY** is missing, OpenAI is unavailable, or the response cannot be validated.

API and Persistence

- FastAPI application with CORS configured for the Vite development server.
- Async SQLAlchemy with SQLite and **aiosqlite**.
- Analysis jobs include the submitted and normalized URL, status, verdict, score, explanation, evidence JSON, error data, and timestamps.
- Screenshot retrieval verifies that the requested job owns the expected artifact before serving it.
- Development OpenAPI and ReDoc endpoints are available through FastAPI when the environment is not production.

Dashboard

- React, TypeScript, Vite, and Tailwind CSS frontend.
- URL input with an in-progress scanning state.
- Safe, Suspicious, and Malicious verdict badges and risk-score bar.
- Human-readable explanation, job metadata, and validation details.
- Browser-capture details, redirect timeline, final URL, and screenshot preview.
- AI assessment details, including confidence, action, model, status, fallback state, and top evidence weights.
- Technical and reputation evidence sections.
- Retry controls for transport errors and persisted failed jobs.
- Responsive layout and a light/dark theme toggle.

Quality and Developer Experience

- Type hints and strict mypy configuration in the backend.
- Ruff linting for backend quality checks.
- Pytest coverage for validation, API lifecycle, browser error mapping, technical analyzers, reputation integrations, and OpenAI response handling.
- Vitest and Testing Library coverage for dashboard rendering, successful scans, failure states, retries, screenshot behavior, and theme toggling.
- Environment-based configuration with **.env.example** templates.
- Standard application logging to stdout.

Architecture

ClickSafe follows clean-architecture dependency direction: framework and external-service code depends on application services; application services depend on domain contracts; domain models remain framework independent.

```

Browser UI (React + TypeScript + Tailwind)
  |
  v
FastAPI routes and schemas
  |
  v
Application services and URL validation
  |
  +--> Domain models and evidence contracts
  |
  +--> Analyzers
  |
  +--> Infrastructure adapters
        |- SQLite repository
        |- Playwright Chromium client
        |- OpenAI Responses API client
        |- VirusTotal client
        `- Google Safe Browsing client

```

Repository Layout

```

clickSafe/
|- backend/
|  |- src/clicksafe/
|  |  |- api/           FastAPI routers and dependency wiring
|  |  |- application/   URL validation and analysis orchestration
|  |  |- analyzers/     One phishing analyzer per file
|  |  |- core/          Settings and logging
|  |  |- domain/        Job, verdict, and evidence contracts
|  |  |- infrastructure/ Browser, database, AI, and reputation adapters
|  |  `- schemas/       Request and response models
|  `- tests/           Backend unit and API tests
|- frontend/
|  `- src/
|     |- components/    Scanner, shell, and result UI
|     |- lib/           Backend API client
|     |- types/         TypeScript analysis contracts
|     `- __tests__/_    Frontend tests
|- docs/
|  `- architecture.md   Architecture documentation
|- README.md            Quick start and current endpoint overview
|- ROADMAP.md           Phase-by-phase implementation plan
|- PROJECT_DETAILS.md   This complete project reference

```

API Contract

Base path: `/api/v1`

Method	Path	Purpose
GET	<code>/health</code>	Returns application health, name, version, and environment.
POST	<code>/analyze</code>	Validates, scans, analyzes, persists, and returns one analysis job.
GET	<code>/analyses?limit=20</code>	Returns recent analysis jobs. The limit range is 1 to 100.
GET	<code>/analyses/{analysis_id}</code>	Returns a saved job by ID.
GET	<code>/analyses/{analysis_id}/screenshot</code>	Returns the analysis PNG when its expected artifact exists.

Analyze Request

```
{
  "url": "https://example.com"
}
```

Analyze Response Shape

```
{
  "id": "uuid",
  "submitted_url": "https://example.com",
  "normalized_url": "https://example.com/",
  "status": "completed",
  "verdict": "Safe",
  "risk_score": 8,
  "explanation": "Human-readable risk explanation.",
  "evidence": {
    "validation": {},
    "browser": {},
    "technical_analysis": [],
    "reputation": [],
    "ai": {},
    "lifecycle": {}
  },
  "error_message": null,
  "created_at": "ISO-8601 timestamp",
  "updated_at": "ISO-8601 timestamp",
  "completed_at": "ISO-8601 timestamp"
}
```

The **evidence** object is intentionally extensible. Current top-level groups are:

- **validation**: normalized URL data and validation result.
- **browser**: capture status, final URL, redirects, artifact paths, HTML size, and truncation.
- **technical_analysis**: standardized analyzer evidence items.
- **reputation**: reputation evidence and per-provider results.
- **ai**: provider state, verdict, score, explanation, confidence, recommendation, and evidence weights.
- **lifecycle**: internal job progress metadata.

Configuration

Copy the root **.env.example** to **.env** for local configuration. Do not commit real API keys.

Variable	Purpose	Default
APP_NAME	API display name	ClickSafe API
APP_ENV	development, test, or production	development
APP_VERSION	Application version	0.1.0
LOG_LEVEL	Server log level	INFO
API_V1_PREFIX	API base path	/api/v1
DATABASE_URL	Async SQLite connection URL	sqlite+aiosqlite:///./clicksafe.db
SQL_ECHO	Emit SQL statements	false
OPENAI_API_KEY	Enables AI verdict generation	empty
OPENAI_MODEL	Responses API model	gpt-5.6
OPENAI_REASONING_EFFORT	Model reasoning setting	medium
OPENAI_TIMEOUT_SECONDS	AI request timeout	30
OPENAI_MAX_OUTPUT_TOKENS	Maximum AI output tokens	1200
VIRUSTOTAL_API_KEY	Enables VirusTotal lookup	empty
GOOGLE_SAFE_BROWSING_API_KEY	Enables Google Safe Browsing lookup	empty
REPUTATION_TIMEOUT_SECONDS	Timeout per reputation provider	10
PLAYWRIGHT_HEADLESS	Run Chromium without a visible	true

Variable	Purpose	Default
	window	
BROWSER_TIMEOUT_MS	Browser navigation timeout	15000
MAX_REDIRECTS	Browser redirect limit	10
MAX_HTML_BYTES	Maximum persisted HTML artifact size	1000000
SCREENSHOT_DIR	PNG artifact directory	data/screenshots
HTML_DIR	HTML artifact directory	data/html
CORS_ORIGINS	Comma-separated allowed frontend origins	local Vite URLs
VITE_API_BASE_URL	Frontend API origin	http://127.0.0.1:8000

Local Development

Backend

```
cd backend
python -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
python -m pip install -e ".[dev]"
python -m playwright install chromium
uvicorn clicksafe.main:app --reload
```

The backend normally listens on <http://127.0.0.1:8000>. Interactive API documentation is at <http://127.0.0.1:8000/docs> in development mode.

Frontend

```
cd frontend
npm install
npm run dev
```

The Vite development server normally listens on <http://localhost:5173>.

Verification Commands

```
# From backend with the virtual environment active
pytest
```

```
ruff check src tests
mypy src

# From frontend
npm run test
npm run build
```

Technology Stack

Area	Technology
Backend API	Python 3.12+, FastAPI, Uvicorn
Validation and settings	Pydantic, pydantic-settings
Database	SQLite, SQLAlchemy async, aiosqlite
Browser automation	Playwright, Chromium
HTML processing	Beautiful Soup
DNS, WHOIS, and TLS	dnspython, python-whois, tldextract, cryptography
Reputation	VirusTotal API v3, Google Safe Browsing v4
AI reasoning	OpenAI Python SDK and Responses API
Frontend	React 19, TypeScript, Vite, Tailwind CSS
Icons	lucide-react
Testing	pytest, pytest-asyncio, Vitest, Testing Library
Static checks	Ruff, mypy, TypeScript compiler

Remaining Roadmap

Phase 8: Production Hardening

Status: **planned**.

- Add request rate limiting to protect the browser and external-service integrations.
- Add request-size controls beyond the current URL field length and HTML artifact limits.
- Add SSRF defenses, including blocking localhost, private, link-local, and otherwise restricted network destinations before browser navigation and after redirects.
- Upgrade logging to structured JSON logs with analysis correlation IDs and without leaking sensitive URL query parameters.
- Add end-to-end tests covering the frontend, API, browser workflow, and key failure modes.
- Add deployment documentation, including production environment settings and operational guidance.

Security and Operational Notes

- **Do not expose the current development scanner publicly yet.** It visits URLs supplied by a user, and SSRF/private-network protection is explicitly scheduled for Phase 8.
- A fresh Playwright browser context improves browser-session isolation but is not a complete network-security boundary. Production deployment should use a dedicated sandboxed worker with restricted egress and resource limits.
- Reputation services receive the URL being checked. Do not submit URLs containing secrets or sensitive query parameters without understanding that third-party disclosure.
- Store `.env`, SQLite databases, screenshots, and captured HTML outside version control.
- Screenshot and HTML artifacts can contain sensitive webpage content. Define retention and deletion policies before production use.
- The backend serves screenshots only after checking that the path matches the analysis ID's expected artifact path.
- Keep API keys server-side. The frontend should only know the backend base URL.

Project Completion Criteria

ClickSafe will be ready to move from a local development tool toward production only when Phase 8 is complete, its security controls have been tested, deployment guidance exists, and the team has reviewed provider licensing, secret management, artifact retention, and network isolation.

Related Documents

- [Quick start and endpoint overview](#)
- [Phase-by-phase roadmap](#)
- [Architecture notes](#)
- [Backend-specific commands](#)